SYSTEM DESIGN & IMPLEMENTATION DIRECTIVE

Phasix — Evolution & Devolution

Full system design notes + Unity 6 step-by-step implementation guide

VERSION	DATE	ENGINE	GDD REF
1.1.0	March 2026	Unity 6 Latest LTS · 2D URP	§3 · §5 · §21 · §25

STATUS

Supersedes GDD §3 Tier Structure

REFERENCE STACK

- Unity 6 ScriptableObjects
- JsonUtility Save System
- UI Toolkit (Runtime)
- BFS / Dijkstra Pathfinding
- Event-Driven Architecture
- Digimon (Web Mechanics)

NAVIGATION

Table of Contents

—	PART 1 — SYSTEM DESIGN

§1	Tier Structure — Full Revision	GDD SYNC REQUIRED
§2	The Evolution Graph	LOCKED
§3	Branch Types	LOCKED
§4	Branch Requirement Framework	LOCKED
§5	Devolution Rules	LOCKED
§6	Fusion System	EXPANDED
§7	Discovery & Fog of War	LOCKED
§8	Evolution Web UI	NEW
—	PART 2 — UNITY 6 IMPLEMENTATION	
§9	Data Layer — ScriptableObjects	PHASE 4
§10	Runtime Layer — Creature State	PHASE 4
§11	Logic Layer — Evaluator & Executor	PHASE 4
§12	Save System	PHASE 4
§13	UI Implementation	PHASE 4
§14	Supporting Interfaces & Stubs	PHASE 4
§15	Editor Tooling	PHASE 4

PART 1

System Design

The complete, authoritative design specification for the Evolution and Devolution system. This supersedes and extends GDD §3 tier structure. All references in the GDD to a T3 ceiling should be updated to reflect this document.

Tier Structure — Full Revision

GDD SYNC REQUIRED

> **GDD SYNC REQUIRED:** The current GDD §3 tier diagram shows T1–T3 as the active range with T4–T5 as future tiers and T6/T7 as Celestial-only. This document supersedes that structure. Natural lines now go T1–T5. T6 and T7 are exclusive to Fusion forms. Update GDD §3 tier strip and §21.4 pacing table accordingly.

Every Phasix exists at a tier level that defines its base power ceiling, skill slot count, and evolution potential. The tier system has two structural categories: natural tiers (T1–T5) and fusion tiers (T6–T7). The two categories are architecturally separate — natural lines cap at T5, and fusion forms occupy T6 and T7 exclusively.

Natural Tier Structure — T1 to T5

All capturable species in the wild begin at T1. Natural progression moves upward tier by tier through the evolution web. Each tier expands the creature's type count, skill tree access, and active slot count.

T1	T2	T3	T4	T5	T6	T7
Base Form	—	—	—	*Transcendent*	*Fusion I*	*Fusion II*
1 Primal type 2 skill trees 2 active slots All species start here	1–2 Primal types 4 skill trees 3 active slots First branch choices	2 Primal types 5 skill trees 4 active slots Exotic branches emerge	3 Primal types 6 skill trees 5 active slots Boss conditionals appear	4 Primal types 7 skill trees 5–7 active slots Natural line ceiling Fusion ingredient tier	Fusion-only Requires two same-tier ingredients (T5+T5) May have forward branches Can be T7 ingredient	Fusion-only Requires two T6 ingredients (T6+T6) May have forward branches Rarest forms in game

Fusion Tier Rules

RULE	DETAIL
Result tier formula	The fusion result tier is always `max(ingredient1.tier, ingredient2.tier) + 1`. A T3 + T4 fusion produces a T5 fusion form.

	A T2 + T2 fusion produces a T3 fusion form. The higher ingredient sets the ceiling.
Same-tier required for T6+	To produce a T6 or higher fusion form, both ingredients must be the same tier. T5 + T5 → T6. T6 + T6 → T7. A T4 + T5 mix caps at T5 — it cannot reach T6 regardless of what the formula would produce. This prevents shortcuts to high-tier fusion forms.
Mixed-tier fusions (T2–T5 results)	Below T6, ingredients can be different tiers. T1 + T2 → T3 fusion. T3 + T4 → T5 fusion. These are valid and intended — allowing fusion experimentation at any stage of progression, not just endgame.
Fusion forms may have forward branches	A fusion form is not inherently terminal. Most will be terminal apex forms by design intent. However, the system fully supports fusion forms having their own forward branches — creating a fusion-origin natural line extending upward. This is a special-case feature. Whether a specific fusion form has branches is a per-form species design decision.
Fusion-origin lines behave like natural lines	If a fusion form has forward branches, those branches and the forms they lead to follow all normal evolution rules. Devolution within a fusion-origin line works normally (history stack pop). Devolution from the fusion origin node itself splits back into the two ingredients.
Fusion forms are not capturable in the wild	Fusion origin forms cannot be encountered as wild enemies — only created by the player. Forms further along a fusion-origin line (T2, T3 of the fusion line) may optionally be wild-capturable; this is a per-species design decision.

Fusion Tier Examples

INGREDIENT 1	INGREDIENT 2	RESULT TIER	SAME-TIER RULE
T1	T2	T3 fusion	Not required below T6 — valid
T2	T2	T3 fusion	Same tier — valid
T3	T4	T5 fusion	Not required below T6 — valid
T4	T5	T5 fusion (not T6)	Mixed tier — caps at T5, cannot reach T6
T5	T5	T6 fusion	Same tier — valid, T6 unlocked

T6	T6	T7 fusion	Same tier — valid, T7 unlocked

> **Why the same-tier gate at T6+?** Without it a player could reach T6 by mixing T5 + T4, bypassing the investment of developing two full lines to T5. The same-tier rule ensures T6 and T7 remain genuine endgame achievements requiring both ingredient lines to be fully developed.

Skill Slot Count by Tier

TIER	SKILL TREES AVAILABLE	ACTIVE SLOTS	NOTES
T1	2	2	Introductory constraint — forces focus
T2	4	3	First tree expansion on evolution
T3	5	4	Midgame flexibility begins
T4	6	5	Loadout identity solidifies
T5	7	5–7	Full expression. Slot count varies by species design.
T6	Inherits from ingredients	Max of both parents	Pending species design — treat as T5-equivalent or higher
T7	Inherits from all four lineage parents	Max of all parents	Pending species design

SECTION 2 · SYSTEM DESIGN

The Evolution Graph LOCKED

The evolution web is a single global directed graph. Nodes are forms. Edges are branches. "Species lines" are not structural containers — they are labels for the common paths through the graph. The graph does not enforce line boundaries.

Graph Architecture

CONCEPT	IMPLEMENTATION
Node	One Phasix form at one tier. Has outgoing forward branches and one naturalParent pointer (null at T1 of its line).
Edge	One branch connecting two nodes. Carries trigger type, requirements, and metadata. Direction is forward (lower tier → higher tier) except via devolution which traverses in reverse.
Natural edge	Connects a form to the next tier of the same species line. The default path. Rendered as a solid line.
Crossover edge	Connects a form on one species line to a node on a different line, always one tier higher. The creature enters the foreign line and navigates it normally from that point. Rendered as a dashed curved line.
Fusion link edge	Connects a T5 node to a T6 fusion form. Requires a second ingredient simultaneously. Not a standard traversal edge — requires co-presence of both ingredients.

Crossover Navigation Rules

Crossover branches allow a creature to leave its natural species line and enter a different line at one tier above its current tier. Once on the foreign line, the creature navigates that line's natural path in both directions.

RULE	DETAIL
Entry is always one tier up	A crossover branch from A2 can only target nodes at tier 3. A creature cannot skip tiers via crossover, nor can it cross to the same or lower tier.
Once on foreign line, navigation is normal	After crossing from A2 to B3, the creature can evolve B3→B4→B5 naturally, or devolve B3→B2→B1, following B line's own paths.
History stack enables full retracing	The creature's personal history stack records every move. Devolution always returns to the exact previous node, including back across the crossover point. This enables A1→A2→B3→B2→B1 and full reverse retracing.
Multiple	A form can have 2–3 crossover branches in addition to its natural

crossovers per form	forward branches. Maximum 5 total forward branches per node (GDD guideline, not hard cap).

Per-Creature Navigation Data

The global graph stores no per-creature state. Each creature instance maintains only two pieces of navigation data: its current node pointer, and a personal history stack of every node it has visited in order. Devolution pops the stack. Evolution pushes the current node onto the stack before moving to the target.

> The history stack is the key design insight. It means the same graph node (e.g. B3) can be arrived at from A2 via crossover or from B2 via natural evolution, and the creature will correctly devolve back to whichever node it actually came from — because that is what is on its personal stack, not what B3's naturalParent pointer says.

SECTION 3 · SYSTEM DESIGN

Branch Types LOCKED

Every edge in the evolution graph has a branch type. The type determines the trigger mechanism and the devolution behavior.

TYPE	TRIGGER	VISUAL	ON DEVOLUTION
Standard	Level floor + up to 3 stat thresholds + up to 2 conditionals. Most branches are this type.	Solid line (natural) or dashed curve (crossover)	Creature returns to previous node. Bond preserved. Skills preserved. Unnamed pool grows. Aptitude increments.
Item-Gated	Standard threshold met AND specific key item consumed on evolution. Item is not a bypass — it is an additional requirement on top of	Solid or dashed with item icon indicator	Creature returns AND item is returned. Player retains the ability to re-evolve this branch by spending the item again.

	normal conditions.		
Crossover	Same as Standard but target node is on a different species line, exactly one tier higher. Tagged `pathType: Crossover` for UI distinction.	Dashed curved line, blends source→target line colors	History stack pop returns to the exact node the creature came from, which may be on a different line.
Fusion Link	Both ingredient creatures present in party. Player initiates fusion. Result tier = max(ing1.tier, ing2.tier) + 1. Both are consumed. For T6+, both ingredients must be same tier. One fusion form is born at the result tier.	Dashed magenta arc converging toward fusion node above	Fusion form splits back into both ingredient creatures exactly as they were at the moment of fusion. If the fusion form had forward branches and the creature evolved further, history stack handles return normally until reaching the fusion origin node, which then triggers the split.

Branch PathType Tag

All branches carry a `pathType` enum used by the UI layer to render correctly. This tag does not affect evaluation logic — it is purely presentational metadata.

PATHTYPE	MEANING
Natural	Stays within the same species line. Default progression path.
Crossover	Exits into a different species line. Rendered distinctly in UI to signal the player is leaving their current line.
FusionLink	Two-ingredient convergence. Requires both creatures to be present. Rendered as converging arcs.

SECTION 4 · SYSTEM DESIGN

Branch Requirement Framework

LOCKED

Every Standard and Item-Gated branch has a requirement profile composed of three layers. All three layers must be satisfied simultaneously for evolution to be available.

The Three Layers

LAYER	GUIDELINE	ROLE
Level Floor	Always present. Low anti-exploit threshold per tier.	Not a real progression gate. Exists only to prevent instant re-evolution after devolving. The floor is low enough that natural play hits it within minutes. All branches at the same tier share the same level floor — stat thresholds create the real differentiation between simple and exotic branches.
Stat Thresholds	Up to 3 stats (guideline). No hard cap.	The primary gate. Both stat layers count: base stats (resets on devolve) and the unnamed pool (never resets). The primary route of any species reaches the easiest branch thresholds organically through natural leveling. Alternate and exotic branches require stats harder to reach: a non-Temper stat, or Aptitude that requires prior devolution cycling.
Conditionals	Up to 2 (guideline). No hard cap.	One-time checks that persist forever across all devolution cycles. Once met, never reset. Boss defeated, region reached, skill tree unlocked, creature captured, item in possession, origin type — all remain checked permanently across every future cycle of this creature.

The Two-Layer Stat System

Every creature has two distinct stat layers that both count toward evolution thresholds.

LAYER	WHAT IT IS	RESETS?
Base Stats	The creature's current form stats. Grows through leveling at this tier. Shaped by species ceiling, Temper direction, personality modifier, and player assignment.	Yes — resets to tier floor on devolution.
Unnamed Pool	Persistent accumulated stat layer representing all previous cycles. Grows on every devolution step, scaled by excess stats above tier floor × current Bond level. Belongs to the creature's entire history, not any one form.	Never. Persists through all forms including fusion.

Valid Conditional Types

TYPE	EXAMPLE
Boss Defeated	Must have defeated Ashenveil (Region 2 boss) at least once
Item In Possession	Must have a Void Shard in inventory (not consumed — just present)
Creature Captured / Befriended	Must have a specific species at 40%+ bond in current roster
Skill Tree Unlocked	Must have unlocked at least one Type G (Aspect) skill tree node
Region Reached	Must have reached Region 3 (Ancient World)
Origin Type	Creature must currently have Corrupted or Hollow Origin

Aptitude as the Exotic Gate

Aptitude is the stat that grows specifically through devolution cycling. A high-Aptitude branch rewards players who have explored the evolution web deeply. A freshly captured creature at the same level as a veteran creature cannot reach high-Aptitude branches — the math enforces the investment requirement without any special rules. Aptitude gates are the primary signal during species design that a branch is exotic or rare.

SECTION 5 · SYSTEM DESIGN

Devolution Rules LOCKED

Devolution is always available with no conditions, no cost, and no time limit. The player can devolve from any form at any time. Multiple devolutions can be chained immediately — the player chooses when to stop.

Core Rules

RULE	DETAIL
One tier per action	Each devolution pops exactly one entry from the history stack. A creature at T5 returns to wherever it came from (T4 on its natural line, or a different tier/line if it arrived via crossover). Cannot skip entries.
History stack navigation	Devolution always returns to the exact node the creature came from — determined by the personal history stack, not the node's naturalParent pointer. This enables full retracing of any path including cross-line journeys.
Bond fully preserved	Bond percentage carries through all devolutions. Bond floor at the last milestone reached is maintained and cannot decrease due to devolution alone.
Skill library preserved	All skills learned at any tier are accessible at all times. A creature at T1 that has previously learned T4 skills can equip and use those skills. No efficacy penalty — the stat system provides natural balancing.
All branches reopen	Devolving reopens all evolution branches at the tier returned to. All branches not previously taken are available again. The full web is explorable over time through cycling.
Met conditionals persist	Boss defeated, region reached, skill tree unlocked — all stay permanently met. The level floor is the only requirement that resets, as it is an anti-exploit measure not a one-time achievement.
Unnamed pool grows on devolve	Each devolution step triggers unnamed pool growth: `excessStats × bondMultiplier`. Excess = how far above the tier's base stat floor the creature developed before devolving. Higher bond = larger pool gain.

Aptitude increments on devolve	Aptitude grows by 1 (pending calibration) on each devolution step. The more the player cycles, the higher Aptitude grows, unlocking progressively more exotic branches over time.
Item-gated branch devolve	Devolving from an item-gated form returns the consumed item alongside the creature. The player retains the ability to re-evolve that branch.
Fusion devolve — splits into ingredients	Devolving from a T6 or T7 fusion form splits it back into its two ingredient creatures. Each ingredient returns as it was at the moment of fusion creation.

Unnamed Pool Growth Scaling

SCENARIO	POOL GAIN	WHY
Devolve immediately after reaching tier minimum	Very small	Barely exceeded tier floor. Little excess development to bank.
Devolve after moderate time at tier	Moderate	Meaningful stat growth above floor. Proportional reward.
Devolve after deep development (overleveled)	Large	Significant excess stats accumulated. Full reward for commitment.
Devolve from T5 after full development	Largest	Highest tier = highest floor = highest excess potential.

SECTION 6 · SYSTEM DESIGN

Fusion System EXPANDED V1.1

Fusion is a creation act. Two ingredient creatures combine and produce one new fusion form at `max(ing1.tier, ing2.tier) + 1`. By default fusion forms are terminal, but the system is fully modular — fusion forms can optionally have their own forward branches, creating a fusion-origin line that extends naturally from the fusion event. Devolution from a fusion origin node splits it back into its two

ingredient parents.

Fusion Event Rules

RULE	DETAIL
Result tier formula	`resultTier = max(ingredient1.tier, ingredient2.tier) + 1`. Applies to all fusion acts regardless of whether ingredients are natural forms or prior fusion forms.
Same-tier required for T6 and above	Producing a T6 or higher fusion form requires both ingredients to be at the same tier. T5+T5 → T6. T6+T6 → T7. A T4+T5 mix caps at T5 and cannot reach T6. Enforced by the evaluator at the time of fusion.
Mixed-tier allowed below T6	Any combination of ingredient tiers is valid when the result would be T5 or lower. T2+T3 → T4 fusion. T1+T4 → T5 fusion. No same-tier restriction at these levels.
Both ingredients are consumed on fusion	The two ingredient creatures cease to exist as separate entities the moment fusion occurs. The new fusion form is born in their place.
Devolution splits back exactly	Devolving from a fusion origin node returns both ingredient creatures exactly as they were at the moment of fusion. No stats, bond, or skills are lost on the ingredient side.
Fusion forms may have forward branches (optional)	A fusion form is not required to be terminal. If species design calls for it, a fusion form can have forward branches — standard, item-gated, or crossover — extending into a fusion-origin natural line. The system accommodates this without special-casing: `forwardBranches` on a fusion node works identically to any other node.
Fusion-origin line devolution	A creature that evolved beyond a fusion origin node devolves normally via history stack pop until it reaches the fusion origin node. Devolving the fusion origin node then triggers the ingredient split. No special logic required — the history stack handles it correctly.
Fusion forms float in the web UI	The fusion origin node is positioned at the midpoint x-coordinate between its two ingredient columns, one row above the higher-tier ingredient. If the fusion form has forward branches, those child nodes render in a new column to the right of all natural lines.
Fusion	Fusion origin nodes and fusion-origin line nodes do not have

forms are self-contained — no crossovers in or out	crossover branches into natural lines, and natural lines cannot crossover into fusion-origin lines.
Fusion origin forms not capturable in the wild	Fusion origin forms can only be created by the player. Forms further along a fusion-origin line may optionally be wild-capturable — a per-species design decision.

Fusion in the Graph

The fusion link edges are attached to the ingredient nodes as outgoing forward branches. The fusion origin node has no `naturalParent` — its parents are the two ingredient nodes stored as `fusionIngredient1` and `fusionIngredient2`. If the fusion form has forward branches, those child nodes have `naturalParent` pointing to the fusion origin normally, and the history stack handles all devolution from that point. No special fusion-specific traversal logic is needed anywhere in the graph system.

SECTION 7 · SYSTEM DESIGN

Discovery & Fog of War LOCKED

Discovery is global and player-wide. When any creature the player controls discovers a form — by personally evolving into it, devolving into it, or encountering it as a wild enemy — that form is permanently revealed for the entire player account. One creature seeing a form means all the player's creatures see it in the web UI.

Three Visibility States

STATE	SPRITE	NAME	REQUIREMENTS	PATH LINES

Hidden	Not shown	Not shown	Not shown	Not shown
Sighted	Silhouette	???	Not shown	Shown
Discovered	Real sprite	Real name	Shown	Shown

Cascade Rule — Sighted

A node becomes Sighted when any of its direct graph neighbours (forward branches or natural parent) is in the Discovered state. Sighted is computed at render time — it is never stored. If discovery state changes, Sighted updates automatically the next time the UI renders.

Discovery Write Sources

- **Evolution or devolution:** The destination node is immediately added to the player's discovered set.
- **Wild encounter:** The moment a wild Phasix of that form appears in an encounter, its form ID is added to the discovered set.
- **Fusion forms:** Fusion origin forms cannot be discovered via wild encounter — only by personally creating the fusion. A fusion origin node shows as Sighted once both of its ingredient parents are Discovered. Forms further along a fusion-origin line (if the fusion form has forward branches) follow standard discovery rules and may be discoverable via wild encounter if the species design permits it.

Plan Mode Visibility

In the evolution web UI plan mode, the player can tap any two nodes to see a path between them. Hidden nodes (not yet Sighted) are still tappable in plan mode — their identity is not shown, but a path can be computed and shown, with undiscovered nodes along the path rendering as `???`. This fulfils the design goal: the player can know a path exists without knowing what forms are on it.

SECTION 8 · SYSTEM DESIGN

Evolution Web UI NEW

The evolution web UI is a zoomable, pannable graph visualisation that replaces the traditional linear evolution menu. It solves the key problem identified with Digimon games: the player had no ability to visualise paths or plan routes through the web. The UI gives the player that ability without hand-holding.

Layout Specification

ELEMENT	POSITION	NOTES
Natural line columns	Left-to-right, one column per species line. Fixed horizontal spacing (~180px between lines).	T1 at bottom row, T5 at top row. Horizontal row guide lines at each tier.
Fusion origin nodes	Float above their two ingredient columns. X position = midpoint of the two ingredient columns. Y position = one row above the higher-tier ingredient row.	Not assigned to any column. Positioned dynamically based on ingredient positions. If the fusion form has forward branches, its child nodes render in a new column appended after all natural line columns.
T7 fusion nodes	Float one full row above the T6 row.	Visually the apex of the entire web.
Tier row labels	Left side of the viewport, fixed. Shows T1–T5, T6 (magenta), T7 (gold).	Sticky at all zoom levels.
Line labels	Below each species line's T1 node.	Colored by line color.

Node Visual Design

FORM TYPE	SHAPE	DISCOVERED	SIGHTED

Natural form	Circle	Colored ring + glow + center dot + tier pips below	Dashed circle silhouette, ??? label
T6 Fusion	Circle (larger, ~1.25× radius)	Split-color arcs (left half = primary ingredient color, right half = secondary). White center dot. Magenta label accent.	Dashed larger circle, magenta tint, ??? label
T7 Fusion	Circle (largest, ~1.4× radius)	Same as T6 but gold accent. Outer pulse ring.	Dashed largest circle, gold tint, ??? label

Edge Visual Design

EDGE TYPE	STYLE	COLOR
Natural	Solid straight line	Gradient from source node color to target node color, 32% opacity
Crossover	Dashed curved arc (slight upward bow)	Gradient source→target, 70% opacity
Fusion Link	Dashed curved arc (stronger arc toward T6/T7 float position)	Source color → magenta (T6) or gold (T7)

Interaction Modes

MODE	BEHAVIOR
Browse (default)	Pan with single touch/drag. Pinch to zoom. Tap a Sighted node to discover it. Tap a Discovered node to see tooltip with name, requirements, and outgoing branches.
Plan Mode	Tap 1: Select origin node (highlights green). Tap 2: Select destination. BFS or Dijkstra computes path. Web dims. Path edges and nodes highlight with step numbers. For fusion destinations, two simultaneous paths show (one to each ingredient).

Pathfinding — Two Modes

MODE	ALGORITHM	EDGE WEIGHTS	FINDS
Shortest	BFS	All edges = 1	Fewest evolution/devolution steps
Easiest	Dijkstra	Natural=1, Crossover=3, FusionLink=6	Path with least difficult requirements (avoids crossovers and fusion links when possible)

PART 2

Unity 6 Implementation

Step-by-step implementation guide for Unity 6 Latest LTS. Each section follows the three-part format: C# script → Inspector setup → tuning guidance. All paths reference the correct Phasix project structure.

SECTION 9 · UNITY 6 IMPLEMENTATION

Data Layer — ScriptableObjects

PHASE 4 · MO 10

The evolution graph is defined entirely as ScriptableObject assets. One global registry holds all nodes. No evolution data lives on individual creature instances — they only hold pointers into this global graph.

Step 1 — Create EvolutionNodeSO

Each node in the graph is one `EvolutionNodeSO` asset. Create one asset per Phasix form.

C# ASSETS/SCRIPTS/EVOLUTION/EVOLUTIONNODESO.CS

```
using UnityEngine;
using System.Collections.Generic;

[CreateAssetMenu(fileName = "EvoNode_", menuName = "Phasix/Evolution/Node
```

```
public class EvolutionNodeSO : ScriptableObject
{
    public string         formID;          // unique string key, e.g. "A3
    public string         displayName;
    public int            tier;            // 1-7
    public string         speciesLine;     // "A", "B", null for fusion f
    public Sprite         formSprite;
    public Color          lineColor;

    [Header("Graph Structure")]
    public List<EvolutionBranchSO> forwardBranches;
    public EvolutionNodeSO    naturalParent;   // null if T1 of line, nul

    [Header("Fusion (leave null/false for natural forms)")]
    public bool            isFusionNode;     // true for fusion origin no
    public EvolutionNodeSO  fusionIngredient1; // required ingredient A
    public EvolutionNodeSO  fusionIngredient2; // required ingredient B
    public bool            fusionHasLine;    // true if this fusion origi
    // Result tier is computed: max(fusionIngredient1.tier, fusionIngredi
    // Set the 'tier' field accordingly when authoring the asset
}
```

⚙ INSPECTOR SETUP — EVOLUTIONNODESO

formID	Unique string. Convention: LineCode + Tier + FormName. e.g. "A3_Ashcrown", "F-AB_Ashbrack"
tier	Integer 1–7. T6 and T7 for fusion nodes only.
speciesLine	Single-letter code ("A"–"T") for natural forms. Leave empty for fusion nodes.
forwardBranches	List of EvolutionBranchSO assets. Drag in all outgoing branches — natural, crossover, and fusion link branches alike.
naturalParent	The node directly below in this line (e.g. A2 for A3). Null for T1 of any line and null for all fusion nodes.
isFusionNode	True for fusion origin nodes only. False for all natural nodes and for nodes that are part of a fusion-origin line but are not the origin itself.

`fusionIngredient1/2`	Only populated on fusion origin nodes. Drag in the two parent ingredient EvolutionNodeSO assets. Result tier = max(ing1.tier, ing2.tier) + 1 — set the tier field accordingly.
`fusionHasLine`	Set to true only on fusion origin nodes that have forward branches (special case). Drives UI layout — when true, the UI allocates a new column for the fusion-origin line's child nodes rather than treating the node as floating.

Step 2 — Create EvolutionBranchSO

Each edge in the graph is one `EvolutionBranchSO` asset. Create one per outgoing branch from each node.

C# ASSETS/SCRIPTS/EVOLUTION/EVOLUTIONBRANCHSO.CS

```
using UnityEngine;
using System.Collections.Generic;

public enum BranchTriggerType { Standard, ItemGated, FusionLink }
public enum BranchPathType    { Natural, Crossover, FusionLink }
public enum ConditionalType   { BossDefeated, ItemInPossession, CreatureC
public enum StatType          { Vitality, Force, Resonance, Guard, Ward,
public enum OriginType        { Wild, Synthetic, Corrupted, Ascended, Hol

[System.Serializable]
public class StatRequirement
{
    public StatType stat;
    public int      minimumValue;
    public bool     countUnnamedPool = true; // always true per GDD
}

[System.Serializable]
public class ConditionalRequirement
{
    public ConditionalType type;
    public string          referenceID;     // boss ID, region ID, species
    public OriginType      requiredOrigin; // only used when type == Orig
}
```

```
[CreateAssetMenu(fileName = "Branch_", menuName = "Phasix/Evolution/Branc
public class EvolutionBranchSO : ScriptableObject
{
    public EvolutionNodeSO            targetNode;
    public BranchTriggerType          triggerType;
    public BranchPathType             pathType;
    public int                         levelFloor;
    public StatRequirement[]           statRequirements;
    public ConditionalRequirement[]    conditionals;
    public ItemSO                      requiredItem;     // null unless I
    public EvolutionNodeSO            fusionPartner;    // null unless Fu
}
```

⚙ INSPECTOR SETUP — EVOLUTIONBRANCHSO

`targetNode`	Drag in the destination EvolutionNodeSO asset.
`triggerType`	Standard for most. ItemGated if a key item is consumed. FusionLink for T5→T6 and T6→T7 branches.
`pathType`	Natural if staying on the same species line. Crossover if exiting to another line. FusionLink for fusion branches.
`levelFloor`	Low anti-exploit value. Values pending NumericalCalibration.md. Set to 1 as placeholder.
`statRequirements`	Up to 3 entries. Each entry: pick stat, set minimum value, leave countUnnamedPool = true.
`conditionals`	Up to 2 entries. Each entry: pick conditional type, set referenceID (boss ID, region ID, species formID, etc.).
`requiredItem`	Leave null for Standard branches. Assign ItemSO asset for ItemGated branches only.
`fusionPartner`	Leave null except on FusionLink branches. Assign the other required ingredient's node.

Step 3 — Create EvolutionGraphSO

One global registry asset that holds every node in the game. This is the single source of truth for the entire web.

C# ASSETS/SCRIPTS/EVOLUTION/EVOLUTIONGRAPHSO.CS

```
using UnityEngine;
using System.Collections.Generic;

[CreateAssetMenu(fileName = "EvolutionGraph", menuName = "Phasix/Evolutio
public class EvolutionGraphSO : ScriptableObject
{
    public List<EvolutionNodeSO> allNodes;

    private Dictionary<string, EvolutionNodeSO> _lookup;

    public EvolutionNodeSO GetNode(string formID)
    {
        if (_lookup == null) BuildLookup();
        _lookup.TryGetValue(formID, out var node);
        return node;
    }

    public List<EvolutionNodeSO> GetNeighbours(EvolutionNodeSO node)
    {
        var result = new List<EvolutionNodeSO>();
        foreach (var branch in node.forwardBranches)
            if (branch.targetNode != null) result.Add(branch.targetNode);
        if (node.naturalParent != null) result.Add(node.naturalParent);
        if (node.isFusionNode)
        {
            if (node.fusionIngredient1 != null) result.Add(node.fusionIng
            if (node.fusionIngredient2 != null) result.Add(node.fusionIng
        }
        return result;
    }

    private void BuildLookup()
    {
        _lookup = new Dictionary<string, EvolutionNodeSO>();
        foreach (var n in allNodes)
            if (n != null) _lookup[n.formID] = n;
    }

    private void OnEnable() => _lookup = null; // clear cache on reload
}
```

⚙ INSPECTOR SETUP — EVOLUTIONGRAPHSO

`allNodes`	Drag every EvolutionNodeSO asset in the project into this list. This is the only asset that needs to know about all nodes — individual nodes do not reference the graph.

> **IMPORTANT:** Create exactly one EvolutionGraphSO asset at `Assets/Data/Evolution/EvolutionGraph.asset`. Reference it from a GameManager or ServiceLocator singleton. Never create multiple instances. The lookup dictionary is rebuilt automatically on domain reload via `OnEnable`.

PROJECT FOLDER STRUCTURE

FOLDER LAYOUT

```
Phasix/Assets/
  Scripts/
    Evolution/
      EvolutionNodeSO.cs
      EvolutionBranchSO.cs
      EvolutionGraphSO.cs
      EvolutionEvaluator.cs
      EvolutionExecutor.cs
      EvolutionPathfinder.cs
  Data/
    Evolution/
      Graph/
        EvolutionGraph.asset          ← one global asset
      Nodes/
        A_Line/
          EvoNode_A1_Ashveil.asset
          EvoNode_A2_Ashmere.asset
          ... etc per natural line
        Fusion/
          EvoNode_F-AB_Ashbrack.asset
          ... etc per fusion form
      Branches/
        A_Line/
          Branch_A1_to_A2.asset
          Branch_A2_to_A3.asset
          Branch_A2_crossover_to_B3.asset
```

```
        ... etc
      FusionLinks/
        Branch_A5_fusionlink_FAB.asset
        Branch_B5_fusionlink_FAB.asset
```

SECTION 10 · UNITY 6 IMPLEMENTATION

Runtime Layer — Creature State

PHASE 4 · MO 10

The per-creature runtime data holds only navigation state and mutable progression values. All graph definition data lives in ScriptableObjects. This separation means save files are small and saves never need to serialize ScriptableObject references — only string IDs.

PhasixRuntimeData

C# ASSETS/SCRIPTS/CREATURES/PHASIXRUNTIMEDATA.CS

```csharp
using System;
using System.Collections.Generic;
using UnityEngine;

[Serializable]
public class PhasixRuntimeData
{
    // ── Identity ──────────────────────────────────────────
    public string     creatureUID;       // GUID — unique per creature in
    public string     currentNodeID;     // formID of current EvolutionNo
    public string     nickname;

    // ── Navigation ────────────────────────────────────────
    public List<string> historyStack = new();
    // Stack of formIDs visited in order. Pop = devolve. Push+move = evol
    // Bottom of stack = origin (T1 of starting line or fusion split poin
```

```
    // ── Stats ──────────────────────────────────────────────
    public int        level;
    public StatBlock  baseStats;         // resets to tier floor on devol
    public int        unnamedPool;       // never resets — grows on each
    public int        aptitude;          // increments on each devolve
    public float      bond;              // 0-1 — preserved through all d

    // ── Origin & Temper ────────────────────────────────────
    public OriginType currentOrigin;
    public TemperType temper;

    // ── Skill Library ──────────────────────────────────────
    public List<string> skillLibraryIDs = new(); // all skills ever learn
    public List<string> equippedSkillIDs = new(); // currently active loa

    // ── Conditionals Met (persist forever) ─────────────────
    public HashSet<string> defeatedBossIDs    = new();
    public HashSet<string> capturedSpeciesIDs  = new();
    public HashSet<string> unlockedSkillTreeIDs = new();
    public HashSet<string> reachedRegionIDs    = new();

    // ── Personality ────────────────────────────────────────
    public string     personalityID;
}

[Serializable]
public class StatBlock
{
    public int vitality, force, resonance, guard, ward, resolve, instinct

    public int Get(StatType stat) => stat switch {
        StatType.Vitality  => vitality,  StatType.Force     => force,
        StatType.Resonance => resonance, StatType.Guard      => guard,
        StatType.Ward      => ward,      StatType.Resolve    => resolve,
        StatType.Instinct  => instinct,  StatType.Aura       => aura,
        _ => 0
    };
}
```

> **HashSet serialization:** Unity's `JsonUtility` does not natively serialize `HashSet<string>`. Convert to `List<string>` before serializing and reconstruct HashSets on load. See §12 Save System for the wrapper pattern.

SECTION 11 · UNITY 6 IMPLEMENTATION

Logic Layer — Evaluator & Executor

PHASE 4 · MO 10–11

Step 1 — EvolutionEvaluator (pure static, no side effects)

C# ASSETS/SCRIPTS/EVOLUTION/EVOLUTIONEVALUATOR.CS

```
using UnityEngine;

public static class EvolutionEvaluator
{
    public static bool CanEvolve(
        EvolutionBranchSO     branch,
        PhasixRuntimeData    creature,
        PlayerInventory      inventory,
        PlayerProgressData   progress)
    {
        if (creature.level < branch.levelFloor)           return false;
        if (!CheckStats(branch, creature))                return false;
        if (!CheckConditionals(branch, creature, inventory, progress)) re
        if (branch.triggerType == BranchTriggerType.ItemGated)
            if (!inventory.HasItem(branch.requiredItem))  return false;
        if (branch.triggerType == BranchTriggerType.FusionLink)
            if (!CheckFusionPartnerPresent(branch, creature, progress)) r
        return true;
    }

    static bool CheckStats(EvolutionBranchSO branch, PhasixRuntimeData c)
    {
        foreach (var req in branch.statRequirements)
        {
            int value = c.baseStats.Get(req.stat);
            if (req.countUnnamedPool) value += c.unnamedPool;
            if (req.stat == StatType.Aptitude) value = c.aptitude;
            if (value < req.minimumValue) return false;
        }
        return true;
    }
```

```
static bool CheckConditionals(
    EvolutionBranchSO  branch,
    PhasixRuntimeData  c,
    PlayerInventory    inv,
    PlayerProgressData prog)
{
    foreach (var cond in branch.conditionals)
    {
        bool met = cond.type switch {
            ConditionalType.BossDefeated      => c.defeatedBossIDs.Co
            ConditionalType.RegionReached     => c.reachedRegionIDs.C
            ConditionalType.CreatureCaptured  => c.capturedSpeciesIDs
            ConditionalType.SkillTreeUnlocked => c.unlockedSkillTreeI
            ConditionalType.ItemInPossession  => inv.HasItem(cond.ref
            ConditionalType.OriginType        => c.currentOrigin == c
            _ => false
        };
        if (!met) return false;
    }
    return true;
}

static bool CheckFusionPartnerPresent(
    EvolutionBranchSO  branch,
    PhasixRuntimeData  creature,
    PlayerProgressData prog)
{
    if (branch.fusionPartner == null) return false;

    // Partner must be present in party or storage at the correct nod
    if (!prog.HasCreatureAtNode(branch.fusionPartner.formID)) return

    // Enforce same-tier rule for T6+ fusion results
    var target = branch.targetNode;
    if (target.isFusionNode && target.tier >= 6)
    {
        int ing1Tier = target.fusionIngredient1?.tier ?? 0;
        int ing2Tier = target.fusionIngredient2?.tier ?? 0;
        if (ing1Tier != ing2Tier)
        {
            Debug.LogWarning($"[Evaluator] Fusion {target.formID} blo
            return false;
        }
    }
```

```
        return true;
    }
}
```

Step 2 — EvolutionExecutor

C# ASSETS/SCRIPTS/EVOLUTION/EVOLUTIONEXECUTOR.CS

```
using UnityEngine;

public static class EvolutionExecutor
{
    // ── EVOLVE ──────────────────────────────────────────────────────
    public static void Evolve(
        PhasixRuntimeData  creature,
        EvolutionBranchSO  branch,
        PlayerInventory    inventory,
        PlayerProgressData progress,
        EvolutionGraphSO   graph)
    {
        // 1. Bank excess stats into unnamed pool before form swap
        BankExcessStats(creature, graph);

        // 2. Consume item if item-gated
        if (branch.triggerType == BranchTriggerType.ItemGated)
            inventory.ConsumeItem(branch.requiredItem);

        // 3. Handle fusion — consume partner creature
        if (branch.triggerType == BranchTriggerType.FusionLink)
            progress.ConsumeCreatureForFusion(branch.fusionPartner.formID

        // 4. Push current node onto history stack
        creature.historyStack.Add(creature.currentNodeID);

        // 5. Move to target node
        creature.currentNodeID = branch.targetNode.formID;

        // 6. Reset base stats to new tier floor
        creature.baseStats = GetTierFloorStats(branch.targetNode, creatur
        creature.level = branch.levelFloor;

        // 7. Register discovery
        progress.discoveredFormIDs.Add(branch.targetNode.formID);
```

```
        EventBus.Raise(new CreatureEvolvedEvent(creature, branch));
    }

    // ── DEVOLVE ──────────────────────────────────────────────────────
    public static void Devolve(
        PhasixRuntimeData  creature,
        PlayerInventory    inventory,
        PlayerProgressData progress,
        EvolutionGraphSO   graph)
    {
        if (creature.historyStack.Count == 0)
        {
            Debug.LogWarning("Devolve called on creature with empty histo
            return;
        }

        var currentNode = graph.GetNode(creature.currentNodeID);

        // 1. Handle fusion split — return ingredient partner
        if (currentNode.isFusionNode)
            progress.RestoreFusionIngredients(currentNode);

        // 2. Bank excess stats before resetting
        BankExcessStats(creature, graph);

        // 3. Grow Aptitude
        creature.aptitude++;

        // 4. Return item if this was an item-gated branch
        var arrivalBranch = FindArrivalBranch(creature, graph);
        if (arrivalBranch != null && arrivalBranch.triggerType == BranchT
            inventory.ReturnItem(arrivalBranch.requiredItem);

        // 5. Pop history stack — return to previous node
        var previousNodeID = creature.historyStack[creature.historyStack.
        creature.historyStack.RemoveAt(creature.historyStack.Count - 1);
        creature.currentNodeID = previousNodeID;

        // 6. Reset base stats to previous tier floor
        var previousNode = graph.GetNode(previousNodeID);
        creature.baseStats = GetTierFloorStats(previousNode, creature.tem
        creature.level = 1;

        // 7. Bond preserved — no change. Skills preserved — no change.
```

```
        EventBus.Raise(new CreatureDevolvedEvent(creature));
    }

    static void BankExcessStats(PhasixRuntimeData c, EvolutionGraphSO gra
    {
        var node    = graph.GetNode(c.currentNodeID);
        var floor   = GetTierFloorStats(node, c.temper);
        int excess  = Mathf.Max(0, c.baseStats.vitality - floor.vitality)
        int growth  = Mathf.RoundToInt(excess * c.bond);
        c.unnamedPool += growth;
    }

    static EvolutionBranchSO FindArrivalBranch(PhasixRuntimeData c, Evolu
    {
        if (c.historyStack.Count == 0) return null;
        var previousNodeID = c.historyStack[c.historyStack.Count - 1];
        var previousNode   = graph.GetNode(previousNodeID);
        return previousNode?.forwardBranches.Find(b => b.targetNode.formI
    }

    static StatBlock GetTierFloorStats(EvolutionNodeSO node, TemperType t
    {
        // Looks up tier floor values from the NumericalCalibrationSO sin
        // NumericalCalibration values are PENDING — see NumericalCalibra
        // Until locked, this returns a zeroed StatBlock so the system co
        var calibration = ServiceLocator.Get<NumericalCalibrationSO>();
        if (calibration == null)
        {
            Debug.LogWarning("[Executor] NumericalCalibrationSO not regis
            return new StatBlock();
        }
        return calibration.GetTierFloor(node.tier, temper);
        // TODO: populate NumericalCalibrationSO.GetTierFloor() after pro
    }
}
```

Step 3 — EvolutionPathfinder

C# ASSETS/SCRIPTS/EVOLUTION/EVOLUTIONPATHFINDER.CS

```
using System.Collections.Generic;
```

```
public static class EvolutionPathfinder
{
    // BFS — fewest steps
    public static List<string> FindShortestPath(
        string fromID, string toID, EvolutionGraphSO graph)
    {
        if (fromID == toID) return new List<string> { fromID };
        var visited = new HashSet<string> { fromID };
        var queue   = new Queue<List<string>>();
        queue.Enqueue(new List<string> { fromID });

        while (queue.Count > 0)
        {
            var path    = queue.Dequeue();
            var current = path[path.Count - 1];
            var node    = graph.GetNode(current);
            if (node == null) continue;

            foreach (var neighbour in graph.GetNeighbours(node))
            {
                if (visited.Contains(neighbour.formID)) continue;
                var newPath = new List<string>(path) { neighbour.formID }
                if (neighbour.formID == toID) return newPath;
                visited.Add(neighbour.formID);
                queue.Enqueue(newPath);
            }
        }
        return null; // no path found
    }

    // Dijkstra — fewest hard requirements
    public static List<string> FindEasiestPath(
        string fromID, string toID, EvolutionGraphSO graph)
    {
        var dist = new Dictionary<string, float>();
        var prev = new Dictionary<string, string>();
        var vis  = new HashSet<string>();

        foreach (var n in graph.allNodes) dist[n.formID] = float.MaxValue
        dist[fromID] = 0;

        while (true)
        {
            string current = null;
            float  best    = float.MaxValue;
```

```
            foreach (var kv in dist)
                if (!vis.Contains(kv.Key) && kv.Value < best) { best = kv
            if (current == null) break;
            if (current == toID) break;
            vis.Add(current);

            var node = graph.GetNode(current);
            foreach (var nb in graph.GetNeighbours(node))
            {
                float w = EdgeWeight(node, nb);
                if (dist[current] + w < dist[nb.formID])
                {
                    dist[nb.formID] = dist[current] + w;
                    prev[nb.formID] = current;
                }
            }
        }

        if (!prev.ContainsKey(toID) && fromID != toID) return null;
        var path = new List<string>();
        var cur  = toID;
        while (cur != null) { path.Insert(0, cur); prev.TryGetValue(cur,
        return path;
    }

    static float EdgeWeight(EvolutionNodeSO from, EvolutionNodeSO to)
    {
        var branch = from.forwardBranches?.Find(b => b.targetNode == to);
        if (branch == null) return 1f; // reverse/devolve edge
        return branch.pathType switch {
            BranchPathType.Natural    => 1f,
            BranchPathType.Crossover  => 3f,
            BranchPathType.FusionLink => 6f,
            _ => 1f
        };
    }
}
```

NATURAL EDGE WEIGHT

1.0

Dijkstra cost

Baseline. Never change — all other weights are relative to this.

CROSSOVER EDGE WEIGHT

3.0

Dijkstra cost

Easiest path prefers natural edges. Adjust upward if crossovers should feel even more

expensive in the planner.

FUSION LINK WEIGHT

6.0

Dijkstra cost

Fusion paths are the most costly in the easiest-path planner, reflecting the real-world difficulty of preparing two ingredient creatures at the required tiers simultaneously.

SECTION 12 · UNITY 6 IMPLEMENTATION

Save System

PHASE 4 · MO 12

Save data stores only IDs and mutable values — never ScriptableObject references. This keeps save files small, forward-compatible, and independent of asset changes. Unity 6's `JsonUtility` handles serialization. Saves write to `Application.persistentDataPath`.

SaveData Structure

C# ASSETS/SCRIPTS/SAVE/SAVEDATA.CS

```
using System;
using System.Collections.Generic;

[Serializable]
public class PlayerProgressData
{
    // Discovery registry — global, player-wide
    public List<string> discoveredFormIDs  = new(); // HashSet serialized

    // World progress
    public List<string> defeatedBossIDs    = new();
    public List<string> reachedRegionIDs   = new();
```

```
    public int          currentRegionIndex;

    // Roster — all creature instances
    public List<PhasixSaveData> roster       = new();
    public List<string>         activePartyUIDs = new(); // up to party m

    // Inventory
    public List<ItemSaveEntry> inventory     = new();

    // Pending fusion creatures (waiting for fusion act)
    public List<string> pendingFusionCreatureUIDs = new();
}

[Serializable]
public class PhasixSaveData
{
    // Flat representation of PhasixRuntimeData for JsonUtility compatibi
    public string   creatureUID;
    public string   currentNodeID;
    public string   nickname;
    public List<string> historyStack        = new();
    public int      level;
    public int      vitality, force, resonance, guard, ward, resolve, ins
    public int      unnamedPool;
    public int      aptitude;
    public float    bond;
    public string   originType;
    public string   temperType;
    public string   personalityID;
    public List<string> skillLibraryIDs     = new();
    public List<string> equippedSkillIDs    = new();
    // Conditionals serialized as lists (JsonUtility does not support Has
    public List<string> defeatedBossIDs     = new();
    public List<string> capturedSpeciesIDs  = new();
    public List<string> unlockedSkillTreeIDs = new();
    public List<string> reachedRegionIDs    = new();
}
```

SaveManager

C# ASSETS/SCRIPTS/SAVE/SAVEMANAGER.CS

```
using System.IO;
```

```
using UnityEngine;

public static class SaveManager
{
    static string SavePath => Path.Combine(Application.persistentDataPath

    public static void Save(PlayerProgressData data)
    {
        string json = JsonUtility.ToJson(data, prettyPrint: true);
        File.WriteAllText(SavePath, json);
        Debug.Log($"[SaveManager] Saved to {SavePath}");
    }

    public static PlayerProgressData Load()
    {
        if (!File.Exists(SavePath))
        {
            Debug.Log("[SaveManager] No save file found. Creating new.");
            return new PlayerProgressData();
        }
        string json = File.ReadAllText(SavePath);
        return JsonUtility.FromJson<PlayerProgressData>(json);
    }

    public static PhasixRuntimeData FromSaveData(PhasixSaveData s)
    {
        return new PhasixRuntimeData
        {
            creatureUID    = s.creatureUID,
            currentNodeID  = s.currentNodeID,
            nickname       = s.nickname,
            historyStack   = s.historyStack,
            level          = s.level,
            baseStats      = new StatBlock { vitality=s.vitality, force=s
                                             guard=s.guard, ward=s.ward
                                             instinct=s.instinct, aura=
            unnamedPool    = s.unnamedPool,
            aptitude       = s.aptitude,
            bond           = s.bond,
            personalityID  = s.personalityID,
            skillLibraryIDs = s.skillLibraryIDs,
            equippedSkillIDs = s.equippedSkillIDs,
            defeatedBossIDs    = new HashSet<string>(s.defeatedBossIDs),
            capturedSpeciesIDs = new HashSet<string>(s.capturedSpeciesIDs
            unlockedSkillTreeIDs = new HashSet<string>(s.unlockedSkillTre
```

```
            reachedRegionIDs    = new HashSet<string>(s.reachedRegionIDs),
        };
    }
}
```

SECTION 13 · UNITY 6 IMPLEMENTATION

UI Implementation

PHASE 4 · MO 12–13

The evolution web UI is implemented as a Unity 6 UI Toolkit runtime panel. The graph is rendered on a Canvas-based custom render using `MeshGenerationContext` or via a zoomable scrollable container with spawned VisualElements per node.

Recommended UI Architecture — Two-Layer Approach

Given the dynamic panning/zooming requirements and the large number of nodes (100+ for a full 20-line web), a hybrid approach is recommended:

- **Edge layer:** Drawn procedurally using `ImmediateModeElement` or a custom Canvas renderer. This handles the hundreds of lines and curves efficiently without spawning GameObjects per edge.
- **Node layer:** Spawned as `VisualElement` objects with absolute positioning. Positioned via the layout data from `EvolutionGraphSO`. Pool nodes that scroll off-screen for performance.

EvolutionWebController — Core MonoBehaviour

C# ASSETS/SCRIPTS/UI/EVOLUTIONWEBCONTROLLER.CS

```
using UnityEngine;
using UnityEngine.UIElements;
using System.Collections.Generic;

public class EvolutionWebController : MonoBehaviour
```

```
{
    [SerializeField] EvolutionGraphSO    graph;
    [SerializeField] UIDocument          uiDocument;
    [SerializeField] VisualTreeAsset     nodeTemplate;

    private PlayerProgressData                      _progress;
    private Dictionary<string, Vector2>              _nodePositions = new(
    private Dictionary<string, VisualElement>        _nodeElements  = new(
    private VisualElement                             _root;
    private VisualElement                             _graphContainer;

    // Plan mode
    private bool           _planMode;
    private string         _planOriginID;
    private string         _planTargetID;
    private List<string>   _planPath;

    void Start()
    {
        _progress = ServiceLocator.Get<PlayerProgressData>();
        _root = uiDocument.rootVisualElement;
        _graphContainer = _root.Q("graph-container");
        ComputeLayout();
        SpawnNodeElements();
        RefreshVisibility();
    }

    void ComputeLayout()
    {
        const float COL_W  = 180f;
        const float ROW_H  = 150f;
        const float PAD_X  = 120f;
        const float PAD_Y  = 100f;
        const float NAT_TIERS = 5f;

        // Natural nodes
        var lineIndex = new Dictionary<string, int>();
        int ci = 0;
        foreach (var node in graph.allNodes)
        {
            if (node.isFusionNode) continue;
            if (!lineIndex.ContainsKey(node.speciesLine))
                lineIndex[node.speciesLine] = ci++;
            float x = PAD_X + lineIndex[node.speciesLine] * COL_W;
            float y = PAD_Y + (NAT_TIERS - node.tier) * ROW_H;
```

```
            _nodePositions[node.formID] = new Vector2(x, y);
        }

        // Fusion nodes — midpoint between ingredients, above their tier
        foreach (var node in graph.allNodes)
        {
            if (!node.isFusionNode) continue;
            var p1 = _nodePositions[node.fusionIngredient1.formID];
            var p2 = _nodePositions[node.fusionIngredient2.formID];
            float rowOffset = node.tier == 6 ? ROW_H : ROW_H * 2f;
            _nodePositions[node.formID] = new Vector2(
                (p1.x + p2.x) / 2f,
                PAD_Y - rowOffset
            );
        }
    }

    void SpawnNodeElements()
    {
        foreach (var node in graph.allNodes)
        {
            var el = nodeTemplate.Instantiate();
            el.style.position = Position.Absolute;
            var pos = _nodePositions[node.formID];
            el.style.left = pos.x; el.style.top = pos.y;
            el.RegisterCallback<ClickEvent>(_ => OnNodeClicked(node.formI
            _graphContainer.Add(el);
            _nodeElements[node.formID] = el;
        }
    }

    public void RefreshVisibility()
    {
        foreach (var node in graph.allNodes)
        {
            var state = GetVisibilityState(node);
            var el    = _nodeElements[node.formID];
            el.RemoveFromClassList("hidden");
            el.RemoveFromClassList("sighted");
            el.RemoveFromClassList("discovered");
            el.AddToClassList(state.ToString().ToLower());
        }
    }

    NodeVisibility GetVisibilityState(EvolutionNodeSO node)
```

```
    {
        if (_progress.discoveredFormIDs.Contains(node.formID))
            return NodeVisibility.Discovered;
        foreach (var nb in graph.GetNeighbours(node))
            if (_progress.discoveredFormIDs.Contains(nb.formID))
                return NodeVisibility.Sighted;
        return NodeVisibility.Hidden;
    }

    void OnNodeClicked(string formID)
    {
        if (_planMode) { HandlePlanTap(formID); return; }
        var state = GetVisibilityState(graph.GetNode(formID));
        if (state == NodeVisibility.Sighted) { DiscoverNode(formID); }
    }

    void DiscoverNode(string formID)
    {
        _progress.discoveredFormIDs.Add(formID);
        RefreshVisibility();
    }

    void HandlePlanTap(string formID)
    {
        if (_planOriginID == null) { _planOriginID = formID; RefreshPlanO
        if (formID == _planOriginID) { _planOriginID = null; RefreshPlanO
        _planTargetID = formID;
        _planPath = EvolutionPathfinder.FindShortestPath(_planOriginID, f
        RefreshPlanOverlay();
    }

    void RefreshPlanOverlay() { /* apply highlighted CSS classes to path
}

public enum NodeVisibility { Hidden, Sighted, Discovered }
```

⚙ INSPECTOR SETUP — EVOLUTIONWEBCONTROLLER

graph	Drag in the single EvolutionGraph.asset from Assets/Data/Evolution/Graph/
uiDocument	Assign the UIDocument component on this GameObject.

nodeTemplate	Assign a UXML VisualTreeAsset that defines the node card. The template should have CSS classes: .hidden, .sighted, .discovered that control sprite/color/visibility.

SECTION 14 · UNITY 6 IMPLEMENTATION

Supporting Interfaces & Stubs

PHASE 4 · MO 10

Several types are referenced throughout the evaluator, executor, and save system that are not defined elsewhere in this document. These stubs define the expected signatures so the evolution system compiles and integrates correctly with the rest of the game.

PlayerInventory

C# ASSETS/SCRIPTS/PLAYER/PLAYERINVENTORY.CS

```
using System.Collections.Generic;
using UnityEngine;

public class PlayerInventory : MonoBehaviour
{
    private Dictionary<string, int> _items = new(); // itemID → quantity

    // Check by ItemSO reference
    public bool HasItem(ItemSO item)
        => item != null && _items.TryGetValue(item.itemID, out int qty) &

    // Check by ID string (used by ConditionalType.ItemInPossession)
    public bool HasItem(string itemID)
        => !string.IsNullOrEmpty(itemID) && _items.TryGetValue(itemID, ou

    public void ConsumeItem(ItemSO item)
```

```
    {
        if (item == null) return;
        if (_items.ContainsKey(item.itemID)) _items[item.itemID]--;
    }

    public void ReturnItem(ItemSO item)
    {
        if (item == null) return;
        if (!_items.ContainsKey(item.itemID)) _items[item.itemID] = 0;
        _items[item.itemID]++;
    }

    public void AddItem(string itemID, int quantity = 1)
    {
        if (!_items.ContainsKey(itemID)) _items[itemID] = 0;
        _items[itemID] += quantity;
    }
}
```

PlayerProgressData — Evolution-Relevant Methods

C# ASSETS/SCRIPTS/PLAYER/PLAYERPROGRESSDATA.CS (EVOLUTION METHODS)

```
using System;
using System.Collections.Generic;
using System.Linq;

// Partial — only the evolution-relevant members shown here.
// Full class also contains region progress, inventory refs, etc.
[Serializable]
public partial class PlayerProgressData
{
    // ── Discovery ──────────────────────────────────────────
    public HashSet<string> discoveredFormIDs = new();

    public void RegisterDiscovery(string formID)
        => discoveredFormIDs.Add(formID);

    // ── Roster ─────────────────────────────────────────────
    public List<PhasixRuntimeData> roster        = new();
    public List<string>            activePartyUIDs = new();

    // Returns true if any creature in roster or storage is currently at
```

```
    public bool HasCreatureAtNode(string formID)
        => roster.Any(c => c.currentNodeID == formID);

    // Called when a fusion act consumes the partner creature.
    // Saves the partner's state frozen at this moment, removes it from a
    public void ConsumeCreatureForFusion(string partnerFormID)
    {
        var partner = roster.FirstOrDefault(c => c.currentNodeID == partn
        if (partner == null)
        {
            UnityEngine.Debug.LogError($"[Progress] ConsumeCreatureForFus
            return;
        }
        // Freeze partner state — store UID for restoration on devolve
        _frozenFusionPartners[partner.creatureUID] = partner;
        roster.Remove(partner);
        activePartyUIDs.Remove(partner.creatureUID);
    }

    // Called on devolve from a fusion origin node. Restores both ingredi
    public void RestoreFusionIngredients(EvolutionNodeSO fusionNode)
    {
        // The fusion creature being devolved is handled by EvolutionExec
        // This method restores the frozen partner creature(s) back to th
        foreach (var kvp in _frozenFusionPartners)
        {
            if (kvp.Value.currentNodeID == fusionNode.fusionIngredient1?.
                kvp.Value.currentNodeID == fusionNode.fusionIngredient2?.
            {
                roster.Add(kvp.Value);
                // Optionally restore to party if space available
            }
        }
        // Clear frozen partners associated with this fusion form
        _frozenFusionPartners.Clear(); // scoped clearing — refine if mul
    }

    // ── Internal fusion state ──────────────────────────────────
    // Not serialized directly — reconstructed from PhasixSaveData on loa
    [NonSerialized]
    private Dictionary<string, PhasixRuntimeData> _frozenFusionPartners =
}
```

NumericalCalibrationSO — Stub

Referenced by `GetTierFloorStats` in the executor. Populate this ScriptableObject once NumericalCalibration.md values are locked after the progression loop calibration phase.

C# ASSETS/SCRIPTS/DATA/NUMERICALCALIBRATIONSO.CS

```
using UnityEngine;
using System.Collections.Generic;

[CreateAssetMenu(fileName = "NumericalCalibration", menuName = "Phasix/Da
public class NumericalCalibrationSO : ScriptableObject
{
    [Header("Tier Floor Stats — populate after progression calibration")]
    public List<TierFloorEntry> tierFloors; // one entry per tier (1-7)

    public StatBlock GetTierFloor(int tier, TemperType temper)
    {
        var entry = tierFloors?.Find(e => e.tier == tier);
        if (entry == null)
        {
            Debug.LogWarning($"[Calibration] No floor entry for tier {tie
            return new StatBlock();
        }
        // Temper-adjusted floor — Edge/Flux/Anchor get slightly differen
        // TODO: apply temper modifier once growth weights are locked
        return entry.baseFloor;
    }
}

[System.Serializable]
public class TierFloorEntry
{
    public int       tier;
    public StatBlock baseFloor;    // minimum stats reset to on devolve —
    public int       levelFloor;   // anti-exploit minimum level — PENDIN
}
```

EventBus — Minimal Stub

The executor fires events on evolve and devolve. The full EventBus implementation is defined in the Technical Directive v0.1.0. This stub shows the two evolution-

specific event types needed.

C# ASSETS/SCRIPTS/CORE/EVOLUTIONEVENTS.CS

```
// Evolution-specific event types — wire into the project EventBus from T

public class CreatureEvolvedEvent
{
    public PhasixRuntimeData  Creature;
    public EvolutionBranchSO  Branch;      // the branch that was taken
    public CreatureEvolvedEvent(PhasixRuntimeData c, EvolutionBranchSO b)
}

public class CreatureDevolvedEvent
{
    public PhasixRuntimeData Creature;
    public CreatureDevolvedEvent(PhasixRuntimeData c) { Creature = c; }
}

public class FormDiscoveredEvent
{
    public string FormID;
    public FormDiscoveredEvent(string id) { FormID = id; }
}

// Subscribers that will want these events:
// — EvolutionWebController: refresh visibility on discovery
// — BondSystem: award bond on devolve/re-evolve cycle (highest bond gain
// — UIManager: trigger evolution animation, sprite swap, VFX
// — SaveManager: auto-save on evolution/devolution
// — AptitudeTracker: increment aptitude counter on devolve event
```

ServiceLocator — Minimal Stub

Referenced by `NumericalCalibrationSO` lookup and by `PlayerProgressData`. The full implementation is in the Technical Directive v0.1.0. Stub shown for completeness.

C# ASSETS/SCRIPTS/CORE/SERVICELOCATOR.CS (STUB)

```
using System.Collections.Generic;
using UnityEngine;
```

```
public static class ServiceLocator
{
    private static Dictionary<System.Type, object> _services = new();

    public static void Register<T>(T service) => _services[typeof(T)] = s

    public static T Get<T>() where T : class
    {
        if (_services.TryGetValue(typeof(T), out var service)) return (T)
        Debug.LogWarning($"[ServiceLocator] Service {typeof(T).Name} not
        return null;
    }
}

// Register on game boot (GameBootstrap.cs or similar):
// ServiceLocator.Register(Resources.Load<NumericalCalibrationSO>("Data/N
// ServiceLocator.Register(FindObjectOfType<PlayerProgressData>());
// ServiceLocator.Register(FindObjectOfType<PlayerInventory>());
```

⚙ WHAT TO REGISTER AT GAME BOOT

NumericalCalibrationSO	Load from Resources or assign via Inspector on a Bootstrap MonoBehaviour. Must be registered before any evolution logic runs.
PlayerProgressData	Instantiated by SaveManager on load. Register immediately after load completes.
PlayerInventory	MonoBehaviour on the Player GameObject. Register in its Awake().
EvolutionGraphSO	Load from Resources or assign via Inspector. Register before any evaluator or executor calls.

SECTION 15 · UNITY 6 IMPLEMENTATION

Editor Tooling

PHASE 4 · MO 13

Building the evolution graph manually by dragging assets in the Inspector is error-prone at scale. A custom Editor window visualising the graph and validating its structure is essential before the roster grows beyond placeholder data.

EvolutionGraphValidator

A simple editor script that validates the graph structure and reports common errors. Run from the Unity menu before any build.

C# ASSETS/EDITOR/EVOLUTIONGRAPHVALIDATOR.CS

```
using UnityEngine;
using UnityEditor;
using System.Collections.Generic;

public class EvolutionGraphValidator
{
    [MenuItem("Phasix/Validate Evolution Graph")]
    public static void Validate()
    {
        var graph = AssetDatabase.LoadAssetAtPath<EvolutionGraphSO>(
            "Assets/Data/Evolution/Graph/EvolutionGraph.asset");
        if (graph == null) { Debug.LogError("EvolutionGraph.asset not fou

        var errors = new List<string>();
        var ids    = new HashSet<string>();

        foreach (var node in graph.allNodes)
        {
            if (string.IsNullOrEmpty(node.formID))
                errors.Add($"Node {node.name} has empty formID");
            if (!ids.Add(node.formID))
                errors.Add($"Duplicate formID: {node.formID}");

            foreach (var branch in node.forwardBranches)
            {
                if (branch.targetNode == null)
                    errors.Add($"Node {node.formID} has branch with null
                if (branch.triggerType == BranchTriggerType.FusionLink &&
                    errors.Add($"Node {node.formID} FusionLink branch mis
                // Validate crossover tier offset
                if (branch.pathType == BranchPathType.Crossover && branch
                    if (branch.targetNode.tier != node.tier + 1)
```

```
                    errors.Add($"Crossover from {node.formID} to {bra
            }

            if (node.isFusionNode)
            {
                if (node.fusionIngredient1 == null || node.fusionIngredie
                    errors.Add($"Fusion node {node.formID} missing one or

                if (node.fusionIngredient1 != null && node.fusionIngredie
                {
                    // Validate result tier formula: must equal max(ing1,
                    int expectedTier = Mathf.Max(node.fusionIngredient1.t
                    if (node.tier != expectedTier)
                        errors.Add($"Fusion {node.formID}: tier is {node.

                    // Enforce same-tier rule for T6+ results
                    if (node.tier >= 6 && node.fusionIngredient1.tier !=
                        errors.Add($"Fusion {node.formID}: T6+ result req
                }

                // Warn if fusionHasLine is false but forward branches ex
                bool hasBranches = node.forwardBranches != null && node.f
                if (hasBranches && !node.fusionHasLine)
                    errors.Add($"Fusion {node.formID} has forward branche
                if (!hasBranches && node.fusionHasLine)
                    errors.Add($"Fusion {node.formID} has fusionHasLine =
            }
        }

        if (errors.Count == 0)
            Debug.Log($"[EvolutionGraph] Validation passed. {graph.allNod
        else
            foreach (var err in errors) Debug.LogError($"[EvolutionGraph]
    }
}
```

Graph Visualiser — Custom Editor Window

Unity 6.2+ ships with an experimental Graph Toolkit package available via the Package Manager. For earlier Unity 6 builds, implement a lightweight custom EditorWindow using UI Toolkit that renders nodes as draggable elements and draws edges using `generateVisualContent`. This does not need to be production-quality — it is a design authoring tool for verifying the graph structure during species roster

creation.

> Unity's experimental GraphView API (in `UnityEditor.Experimental.GraphView`) is usable for this purpose. It is marked experimental and subject to change, but it is appropriate for internal editor tooling. Alternatively, the mockup web visualisation built in HTML/JS during design exploration can serve as the primary graph authoring reference — the Unity validator script above is sufficient to catch structural errors at build time.

Build Order — Phase 4 Implementation Sequence

1	Create `EvolutionNodeSO`, `EvolutionBranchSO`, and `EvolutionGraphSO` script files. Confirm they appear in the Asset Create menu.
2	Create one `EvolutionGraph.asset` at the designated path.
3	Create placeholder node assets for `PLACEHOLDER_A`, `PLACEHOLDER_B`, `PLACEHOLDER_C` (defined in SpeciesRoster.md) with 2 branches each. Wire into the graph.
4	Implement `PhasixRuntimeData` and `StatBlock`. Verify serialization.
5	Implement `EvolutionEvaluator`. Unit test against placeholder nodes.
6	Implement `EvolutionExecutor` — evolve and devolve paths. Verify history stack push/pop and unnamed pool growth.
7	Implement `EvolutionPathfinder` — BFS and Dijkstra. Verify correctness on placeholder graph.

8. Implement `SaveManager`. Verify save/load round-trip preserves all fields including HashSet reconstruction.

9. Implement `EvolutionGraphValidator` editor script. Run against placeholder graph — expect zero errors.

10. Implement `EvolutionWebController` and UI Toolkit runtime panel. Placeholder sprites only at this stage.

11. Wire evolution events (`CreatureEvolvedEvent`, `CreatureDevolvedEvent`) into the game loop via EventBus.

12. Expand placeholder graph to full roster once SpeciesRoster.md is populated (Phase 5).

> **REMINDER — Don't populate content yet:** SpeciesRoster.md is explicitly a Phase 5 task. Build the framework with the three placeholder species. Every content gap should have a `// TODO: populate after species roster design phase` comment. The validator will catch structural errors; content gaps are expected and intentional at this stage.